

Path Planning Task Documentation

October 5, 2026

1 Introduction

This document explains how I built the path planning algorithm for the FSAI-style cone track. The goal was to take the car's current position and a few cones (blue on the left, yellow on the right) and generate a drivable path forward. My code works whether there are zero, one, or two cones on each side, and it also handles cases where there are three cones on just one side.

2 Algorithm Choices

Before writing the code, I thought about a few different ways to solve this problem. Here is a quick look at the options and why I chose the one I did.

2.1 1. Artificial Potential Fields

This idea treats the car like it's pulled forward by a magnet, while the cones push it away like repelling magnets. Blue cones push the car to the right, and yellow cones push it to the left.

- **Pros:** It easily handles any number of cones without needing much extra code.
- **Cons:** It's hard to tune. If the pushing forces aren't set exactly right, the car's path might wobble a lot, or the car could get stuck if a cone is right in front of it.

2.2 2. Delaunay Triangulation and Voronoi Diagrams

This is a common way to find a path right down the middle of obstacles. It connects all the cones into triangles and finds the exact center lines between them.

- **Pros:** It's very mathematically accurate and creates a path that stays as far away from cones as possible.
- **Cons:** It's way too complicated to code from scratch for this assignment. Plus, when there are only 1 or 2 cones on the track, the math can break down because you can't really form triangles.

2.3 3. Geometric Midpoints and Shifting (Chosen Approach)

This approach looks at the track from the car's perspective. It draws lines between the cones on the left and the cones on the right. If cones on one side are missing, it just takes the side it can see and shifts the path over by a safe distance.

- **Pros:** It's simple, fast, and rarely breaks. It fits the assignment's hint to "keep it simple." It also naturally solves the problem of having 3 cones on only one side.
- **Cons:** It might create sharp turns if the cones are placed weirdly, but adding a little bit of path smoothing fixes this.

3 How the Code Works

I went with the Geometric Midpoints approach. Here is how the code in `src/path_planning.py` works step by step:

3.1 Looking from the Car's Point of View

First, I convert all the cone coordinates into the car's "local frame". This just means moving the math so the car is always at (0,0) and pointing straight ahead along the X-axis.

$$\begin{aligned}x_{local} &= (x_{world} - x_{car}) \cos(-\psi) - (y_{world} - y_{car}) \sin(-\psi) \\y_{local} &= (x_{world} - x_{car}) \sin(-\psi) + (y_{world} - y_{car}) \cos(-\psi)\end{aligned}$$

Here, ψ is the car's yaw (heading). Once we do this, blue cones are always at a positive Y (left), and yellow cones are always at a negative Y (right).

3.2 Finding the Track Center

For every step the car takes forward (along the X-axis), the code guesses where the left and right boundaries are by drawing straight lines between the cones.

Then it finds the center of the path:

- **If both sides have cones:** Center $Y = \frac{Y_{blue} + Y_{yellow}}{2}$
- **If only blue (left) is seen:** Center $Y = Y_{blue} - \text{safe_offset}$
- **If only yellow (right) is seen:** Center $Y = Y_{yellow} + \text{safe_offset}$
- **If no cones are seen:** Center $Y = 0$ (just drive straight)

3.3 Smoothing the Path

To make sure the car starts driving smoothly from its exact current position, I added a blending function. It forces the path to start at $Y = 0$ and smoothly curve into the track center over the first 2 meters. Finally, I average out the points (using a moving average) to get rid of any sharp corners before converting the coordinates back to the global map.

4 What about three cones?

Because my code draws straight lines between cones based on how far forward they are, it doesn't care how many cones there are. If there are three blue cones, it just connects them into a longer boundary line. Then, it shifts the path to the right by the safe offset.

To test this, I added Scenario 21 (three blue cones) and Scenario 22 (three yellow cones) in `src/scenarios.py`. The car perfectly followed the curve of the three cones at a safe distance.

5 Assumptions and Limitations

5.1 Assumptions

- **Track Width:** I assumed the track is about 2.5 meters wide, so the safe offset distance from a single cone is 1.25 meters.
- **Car Direction:** I assumed the car is at least roughly pointing in the direction of the track.

5.2 Limitations

- **Sharp Hairpin Turns:** Because the code plans the path by moving forward along the car's X-axis, it will struggle if the track takes a very sharp U-turn back towards the car.
- **Steering Limits:** The code creates a path, but it doesn't check if a real car's steering wheel can actually turn that sharply.